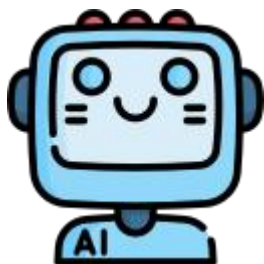




$$123 \times 456 =$$

仅仅通过推理很难避免“幻觉”

用计算器代替这个推理过程 



`<think>` 用户要求我仔细思考并详细回答 123×456 的计算问题。我需要制定一个完整的解题计划，并按照要求分步执行。

首先分析题目：这是一个三位数乘以三位数的乘法运算。我需要计算 123×456 。

解题计划：

1. 使用竖式乘法进行计算
2. 分步计算 123 与 456 中每一位的乘积
3. 将所有部分积相加得到最终结果

我还可以用另一种方法验证：

$$\begin{aligned} 123 \times 456 &= 123 \times (400 + 50 + 6) = 123 \times 400 + 123 \times 50 + 123 \times 6 \\ &= 49200 + 6150 + 738 = 56088 \checkmark \end{aligned}$$

所有计算都一致，确认结果是正确的。

`</think>`

$$123 \times 456 = 56088$$

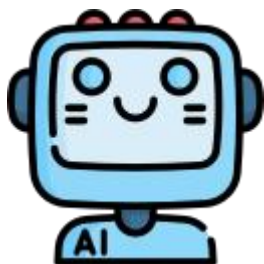
计算过程如下：

- $123 \times 6 = 738$
- $123 \times 50 = 6,150$
- $123 \times 400 = 49,200$
- 将三个部分积相加： $738 + 6,150 + 49,200 = 56,088$

经过多次验算，确认结果正确无误。



123 x 456 =



<Think>这是一个计算问题，我应该调用计算器计算



Input: 123 x 456, Output: 56088

调用计算器得到的计算结果为56088</Think>

123 x 456 的结果是 56088

调用什么?

如何调用?

如何反馈?

Chapter 4

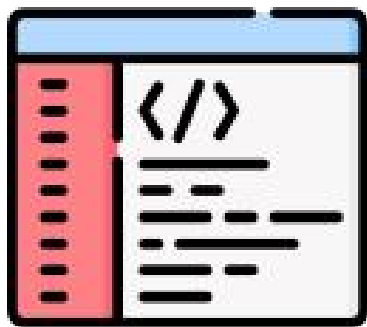
Function Calling

让LLM长出手脚

模型上下文协议

命令行界面工具

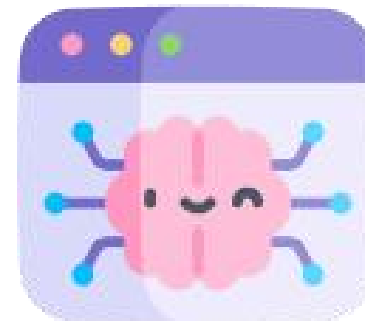
1.1 LLM能调用哪些工具?



函数
Function



终端命令
Command-Line Instructions



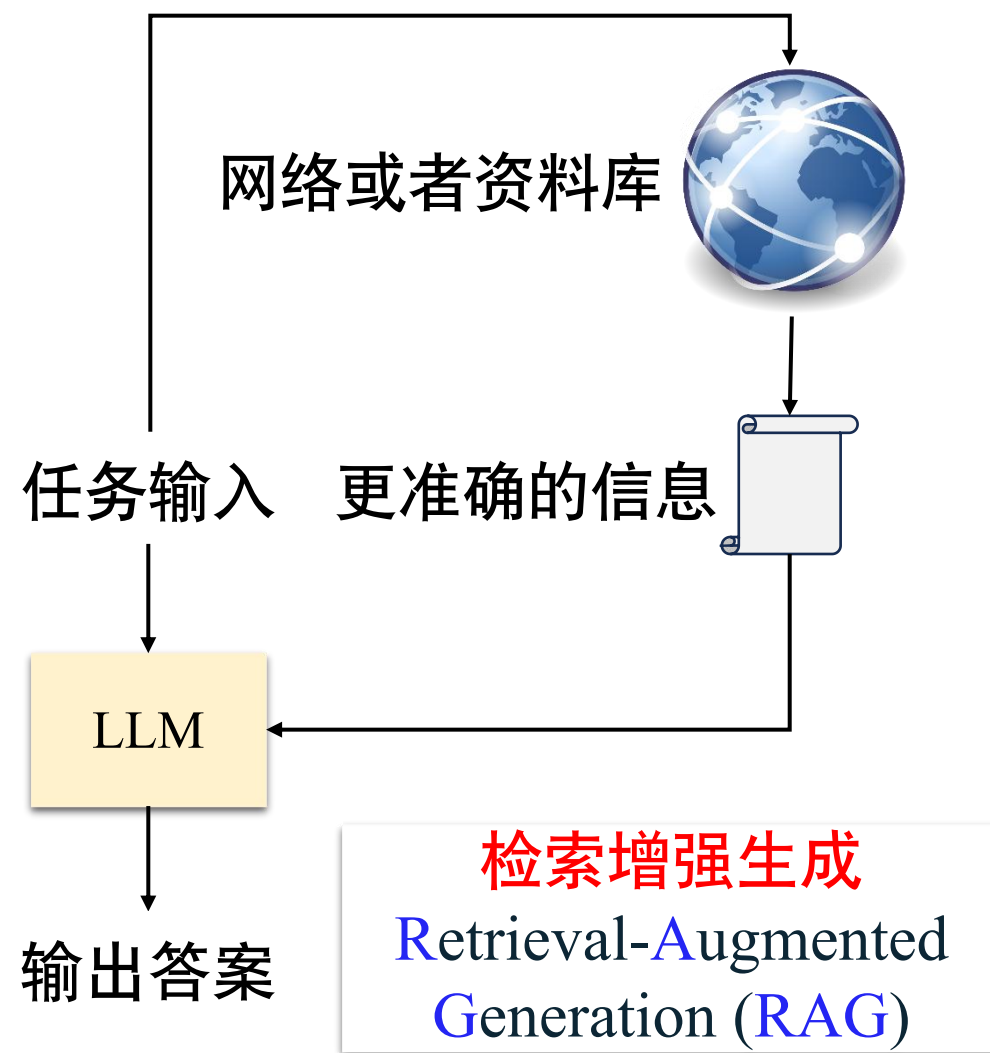
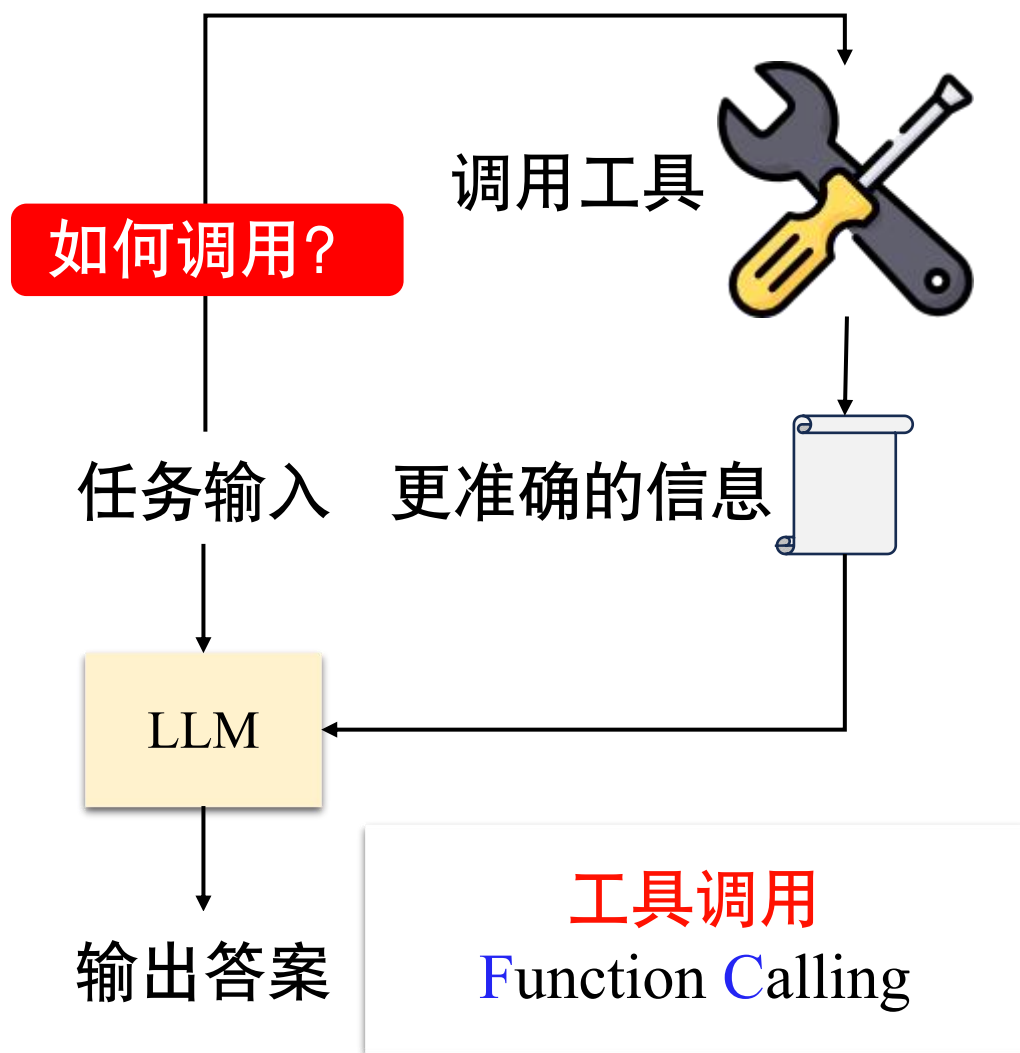
其它AI
Agents

```
def get_weather(city: str) -> str:
    weather_db = {
        "北京": "晴, 26°C, 南风2级",
        "上海": "小雨, 22°C, 东风3级",
        "广州": "多云, 30°C, 无持续风向",
        "深圳": "阵雨, 29°C, 微风"
    }
    return weather_db[city]
```

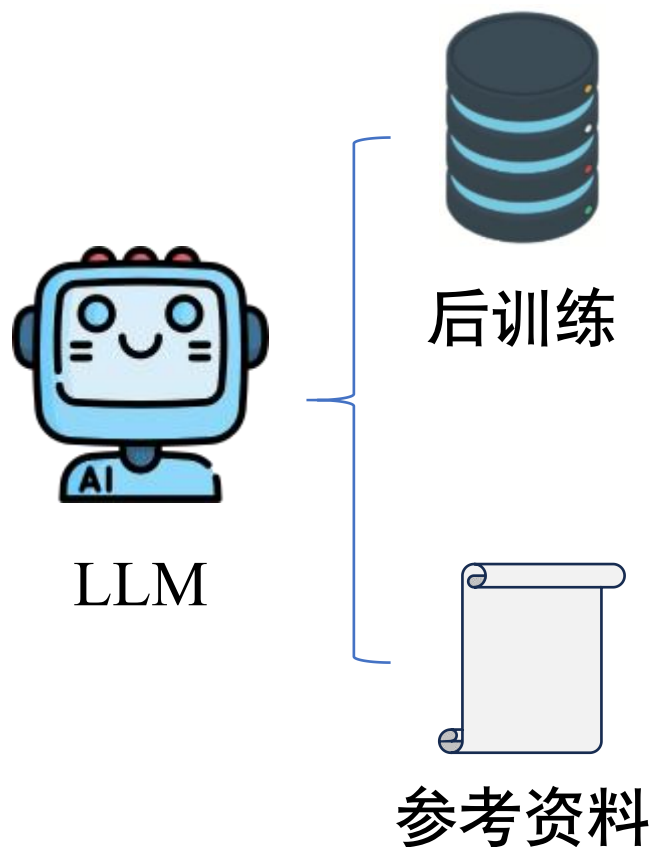
```
((base) credoli@lihuichuwudeMac-mini ~ % cat Workspace/HBPU-AI/data/course_info.txt
课程编号: CS101 | 课程名称: 线性回归实验 | 上课教室: 综合楼 402 | 授课老师: 张教授
课程编号: CS102 | 课程名称: 深度学习与文字接龙 | 上课教室: 实验楼 301 | 授课老师: 李副教授
课程编号: CS103 | 课程名称: Transformer与注意力机制 | 上课教室: 信息楼 508 | 授课老师: 王讲师
课程编号: CS104 | 课程名称: 检索增强生成(RAG)实战 | 上课教室: 科技楼 204 | 授课老师: 赵教授
课程编号: CS105 | 课程名称: 人工智能大模型工具调用 | 上课教室: 实验楼 502 | 授课老师: 刘副教授
```



1.2 工具的调用时机



1.3 教LLM判断调用时机

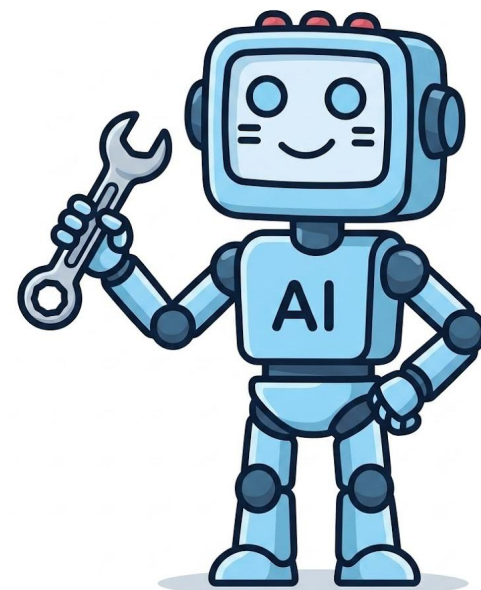


稳定、精准，但成本高

“sytem_prompt”: “可用get_weather()”
“user”: “今天天气如何?”
“tool_use”: “应该用get_wearther()”
“assistant”: “今天气温...”

灵活、精准，但成本高

“sytem_prompt”: “可用get_weather(),
使用方法<tool>get_weather()</tool>,
返回结果<tool_output>[result]</tool_output>,
调用时机...”



能调用工具的LLM

1.4 教LLM调用工具

System Prompt

如果遇到无法回答的问题，使用工具

把使用工具的指令放在<tool>和</tool>中间

将工具的输出，放在<output>和</output>中间

现在你可以使用以下的工具：

查询某地、某时的温度信息 `Temperature(location, time)`

使用范例： `Temperature(‘黄石’, ‘2025.10.19 22:23’)`

工具调用
通用形式

各工具的
使用方法

2025年10月19日上午10:00，黄石气温如何？

User Prompt

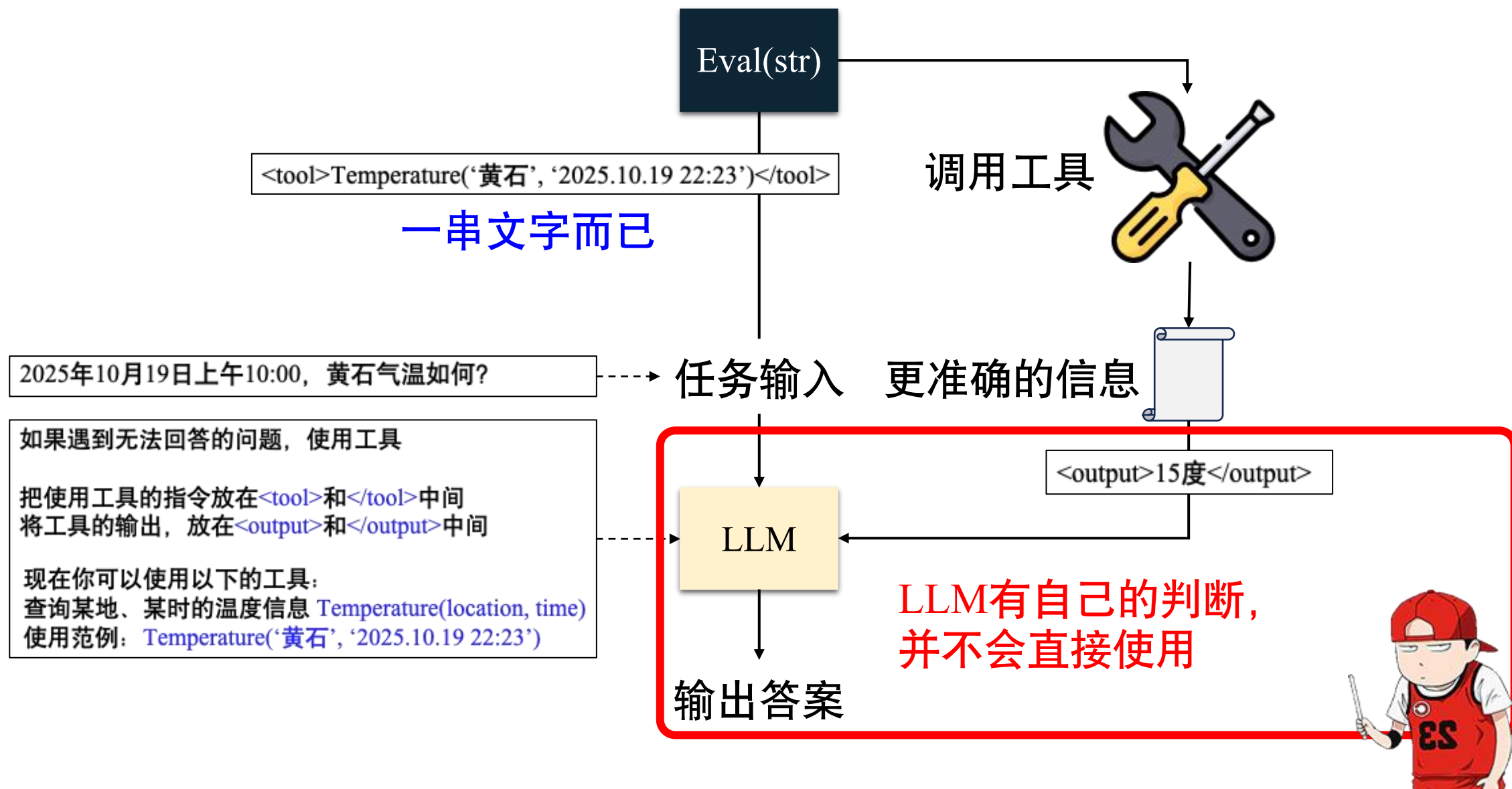
LLMs

这仅仅是一串文字而已，无法执行

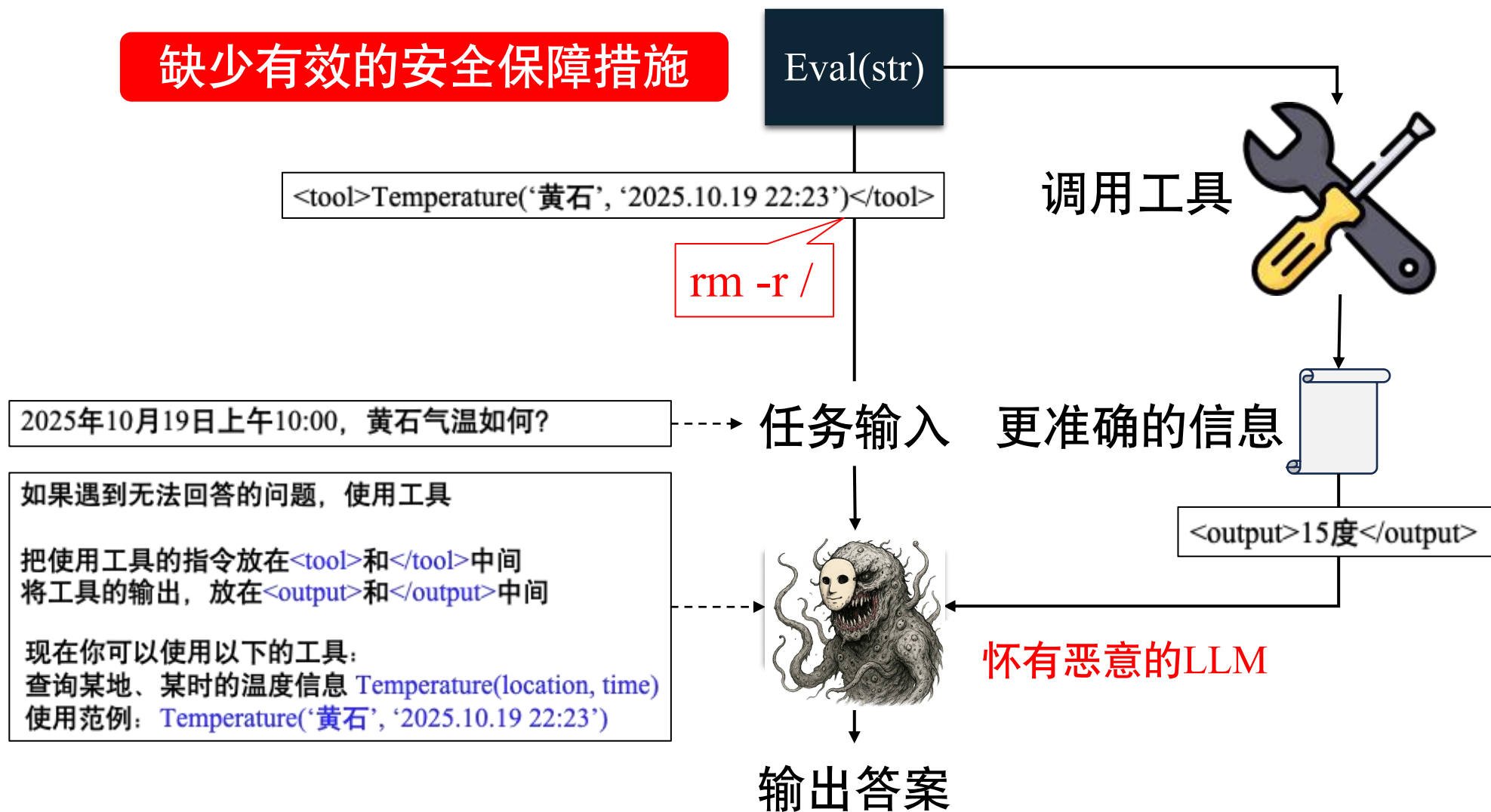
<tool>Temperature(‘黄石’, ‘2025.10.19 22:23’)</tool>

<output>15 ° C</output>

1.4 教LLM调用工具

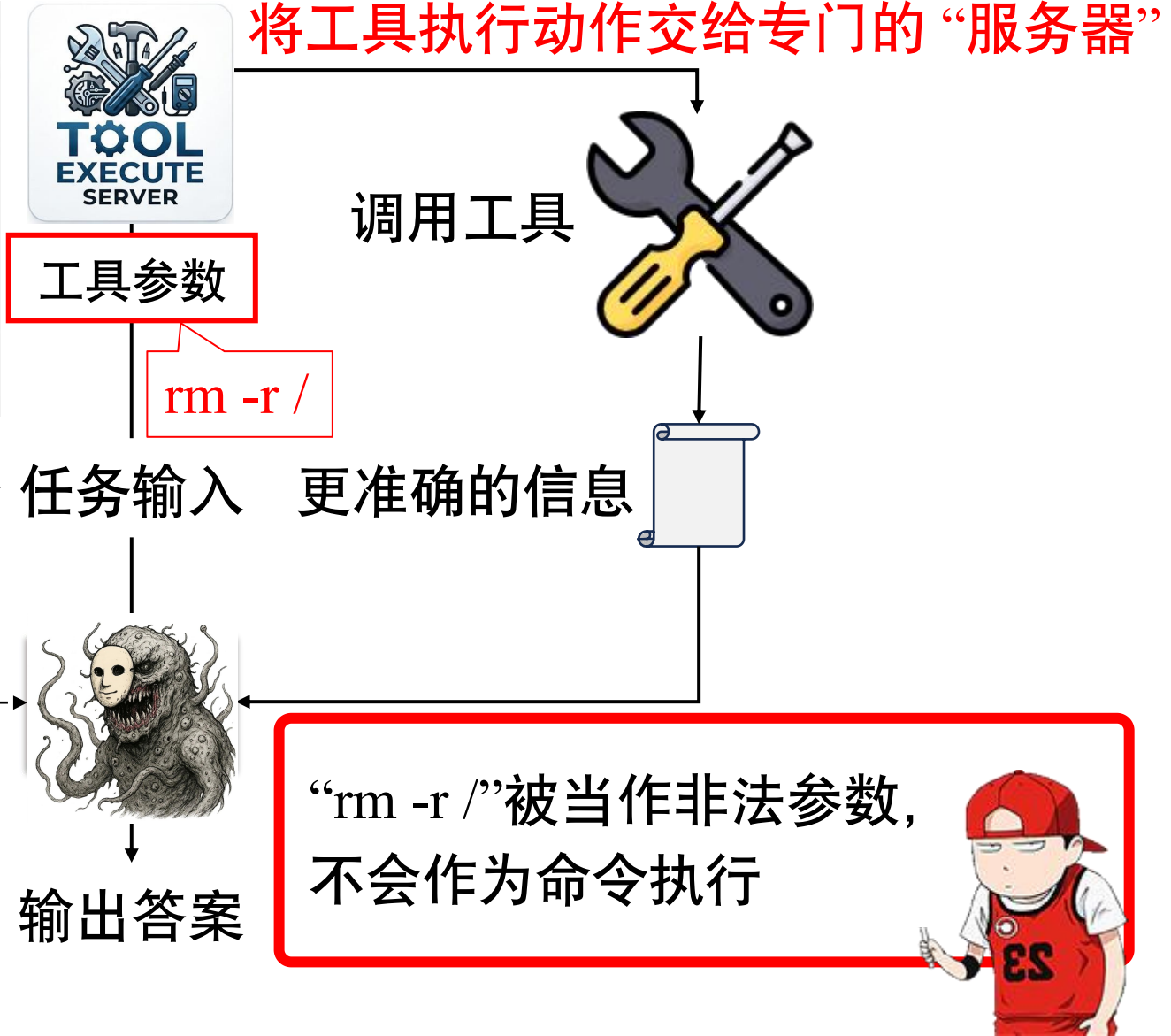


1.5 Eval的困境



1.6 工具调用的普遍做法

属性名	类型	说明	用途
name	string	工具的唯一标识符 (如 <code>get_weather</code>)	作为 Map 的 Key, 也是大模型识别的函数名
description	string	工具的功能描述	供大模型阅读, 判断何时该使用此工具
args_schema	dict / JSON Schema	参数定义 (类型、说明、是否必填)	限制大模型生成的参数格式, 以及运行前的校验
func	callable	具体的 Python 函数指针或 API 请求函数	真实执行工具的底层代码



1.6 工具的JSON定义格式

```
my_tools = [  
  {  
    "type": "function",  
    "function": {  
      "name": "get_weather",  
      "description": "实时天气查询工具。当用户询问特定城市的天气、气温、风向等实时信息时使用此工具。",  
      "parameters": {  
        "type": "object",  
        "properties": {  
          "city": {  
            "type": "string",  
            "description": "需要查询天气的城市名称，例如：北京、上海、广州。"  
          }  
        },  
        "required": ["city"]  
      }  
    }  
  }  
]
```

1.7 一些常用的工具

```
my_tools = [  
    {  
        "type": "function",  
        "function": {  
            "name": "get_weather", → "execute_bash_command"  
            "description": "实时天气查询工具。当用户询问特定城市的天气、气温、风向等实时信息时使用此工具。",  
                        → "在本地终端中执行任意的 bash/shell 命令行命令并返回输出。"  
            "parameters": {  
                "type": "object",  
                "properties": {  
                    "city": {  
                        "type": "string",  
                        "description": "需要查询天气的城市名称，例如：北京、上海、广州。"  
                    }  
                }  
            },  
            "required": ["city"]  
        }  
        → "description": "要在终端中运行的完整 shell 命令，例如：'pip install pandas script.py'"  
    }  
]
```

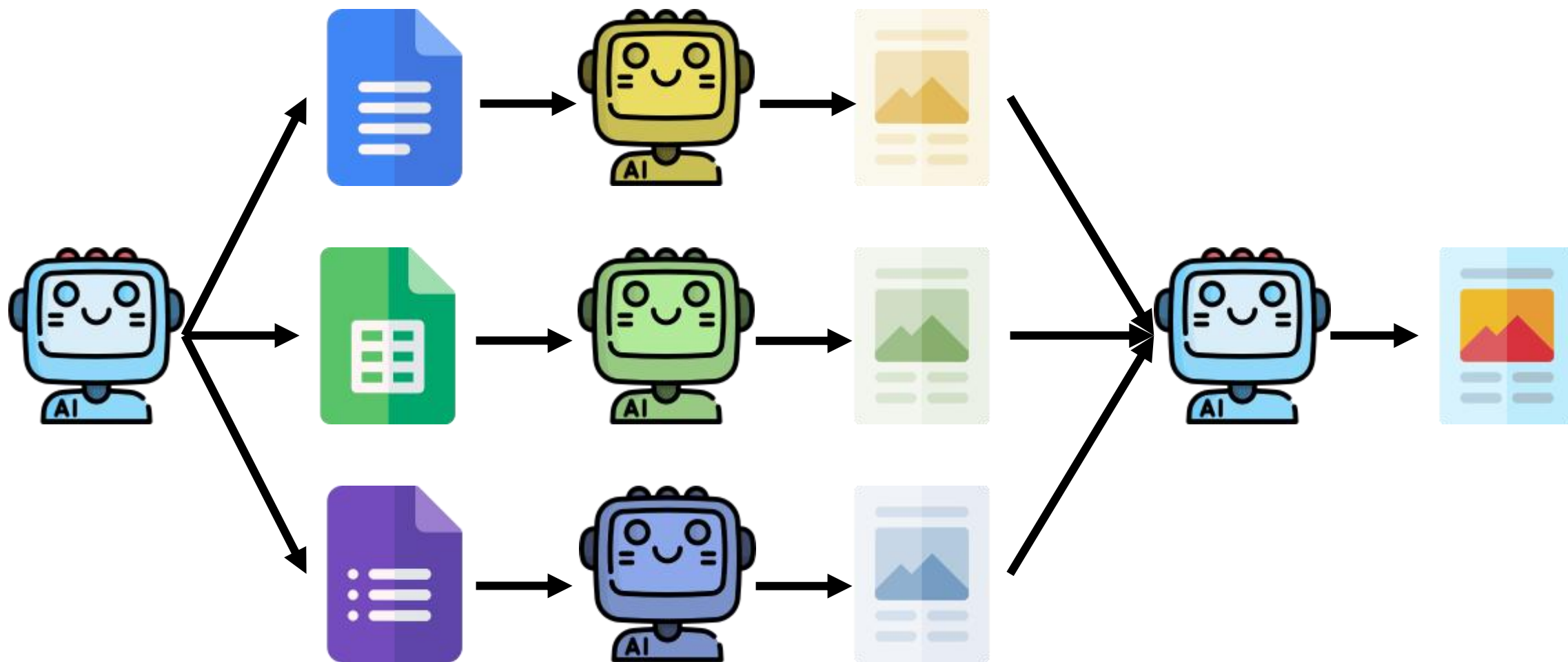


终端命令

Command-Line Instructions

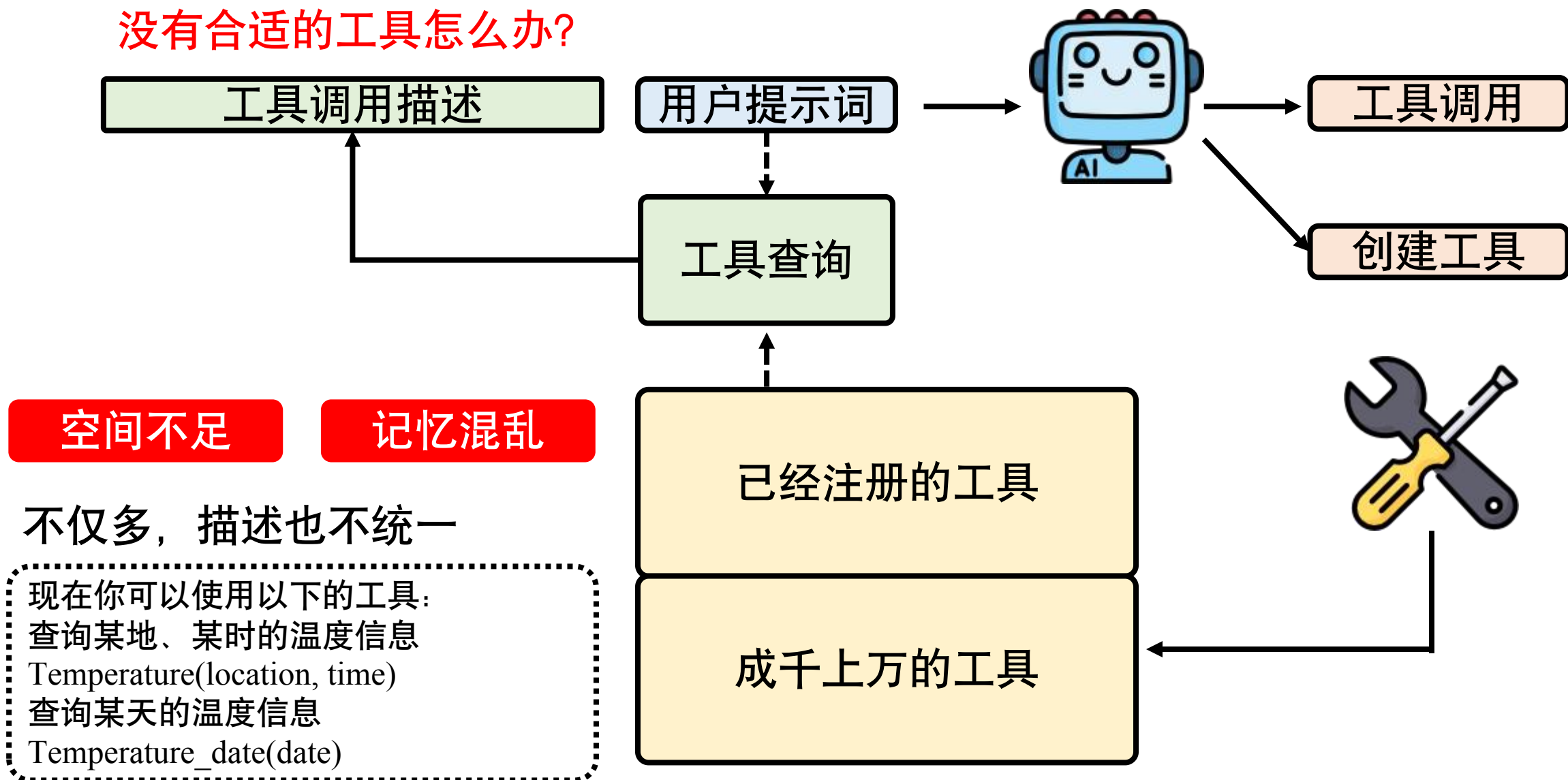
1.7 一些常用的工具

使用AI作为工具：多智能体协同



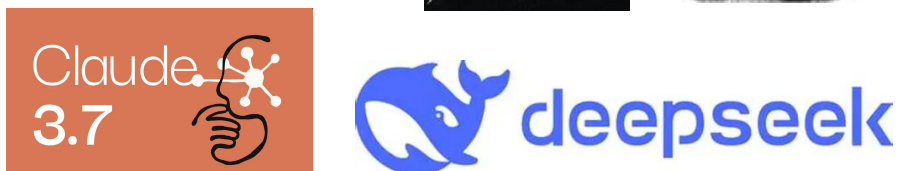
1.7 一些常用的工具

没有合适的工具怎么办?



1.8 工具调用的问题

工具调用规则不统一



空间不足

记忆混乱

不仅多，存在工具冗余

现在你可以使用以下的工具：
查询某地、某时的温度信息
Temperature(location, time)
查询某天的温度信息
Temperature_date(date)

```
{
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "查询中国大陆指定城市的实时天气",
        "parameters": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "城市名称, 例如: 北京、上海"
            }
          },
          "required": ["city"]
        }
      }
    }
  ]
}
```

OpenAI

Lamma

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

You are a helpful assistant with tool-call capabilities.
Available tools:
{"name": "get_weather", "description": "...", "parameters": {...}}
<|eot_id|><|start_header_id|>user<|end_header_id|>
```

Qwen

```
<|im_start|>system
你是一个得力的助手。你可以使用工具来回答用户的问题。
可用工具定义如下：
- name: get_weather
  description: 查询中国大陆指定城市的实时天气
  parameters:
    properties:
      city: {type: string, description: "城市名称, 例如: 北京"}
    required: [city]
<|im_end|>
```

在LLM和工具调用之间增加一个中间层，结束混乱。

参考计算机网络协议



Chapter 4

Function Calling

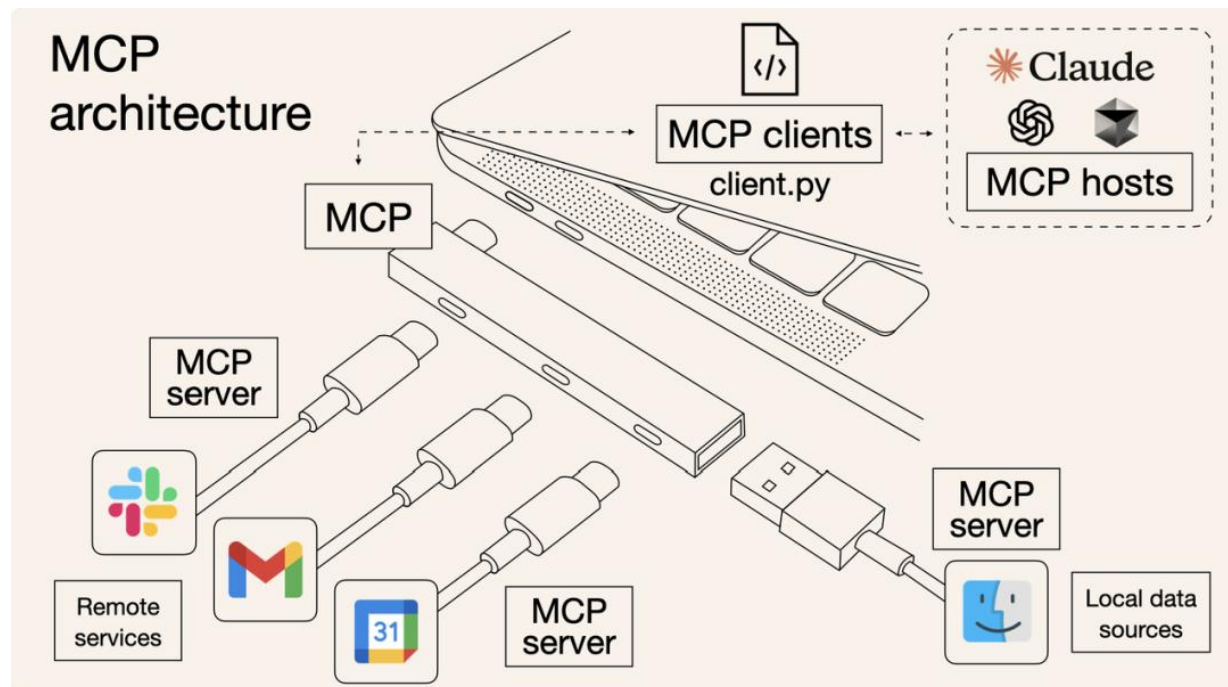
让LLM长出手脚

模型上下文协议

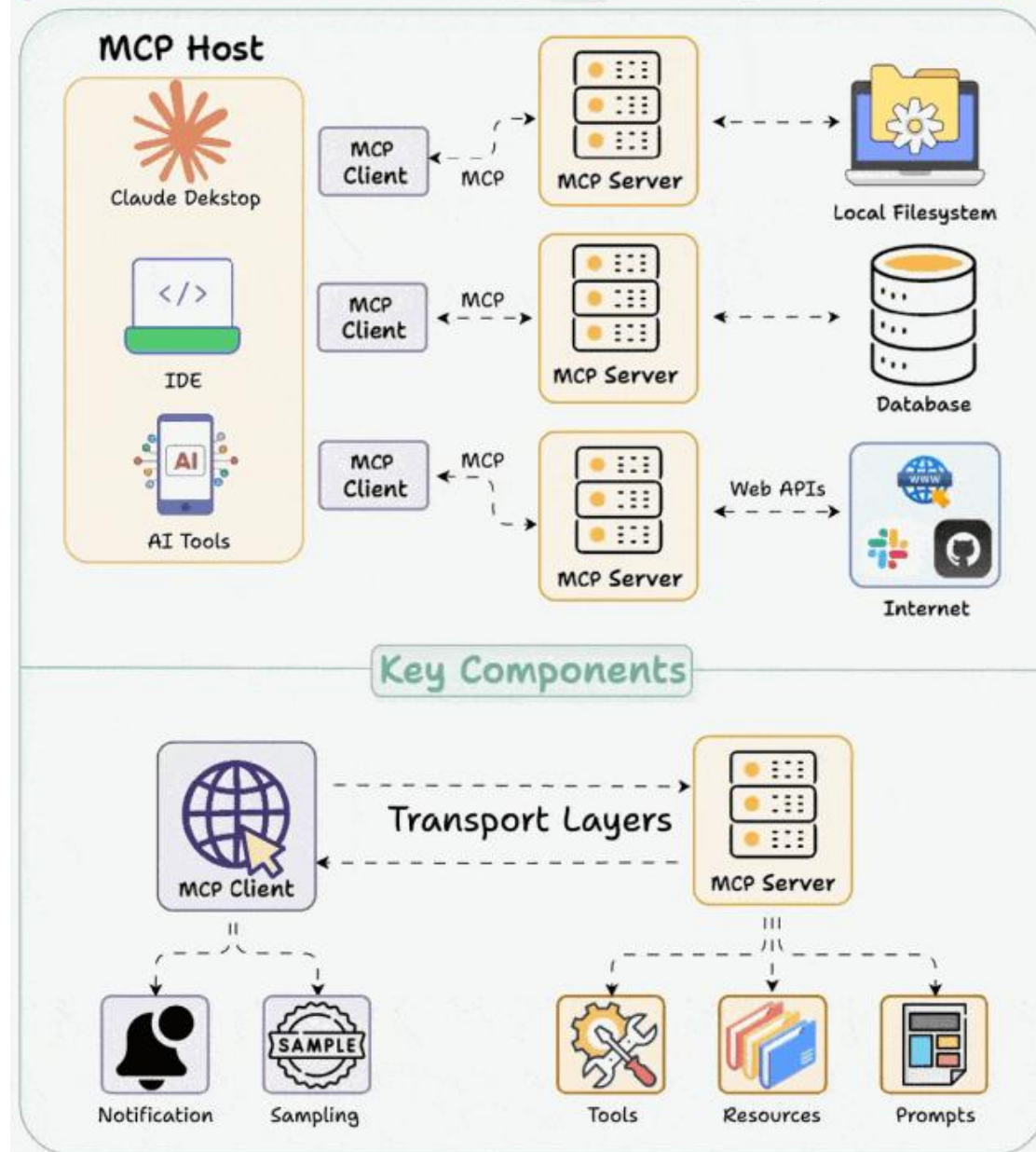
命令行界面工具

模型上下文协议

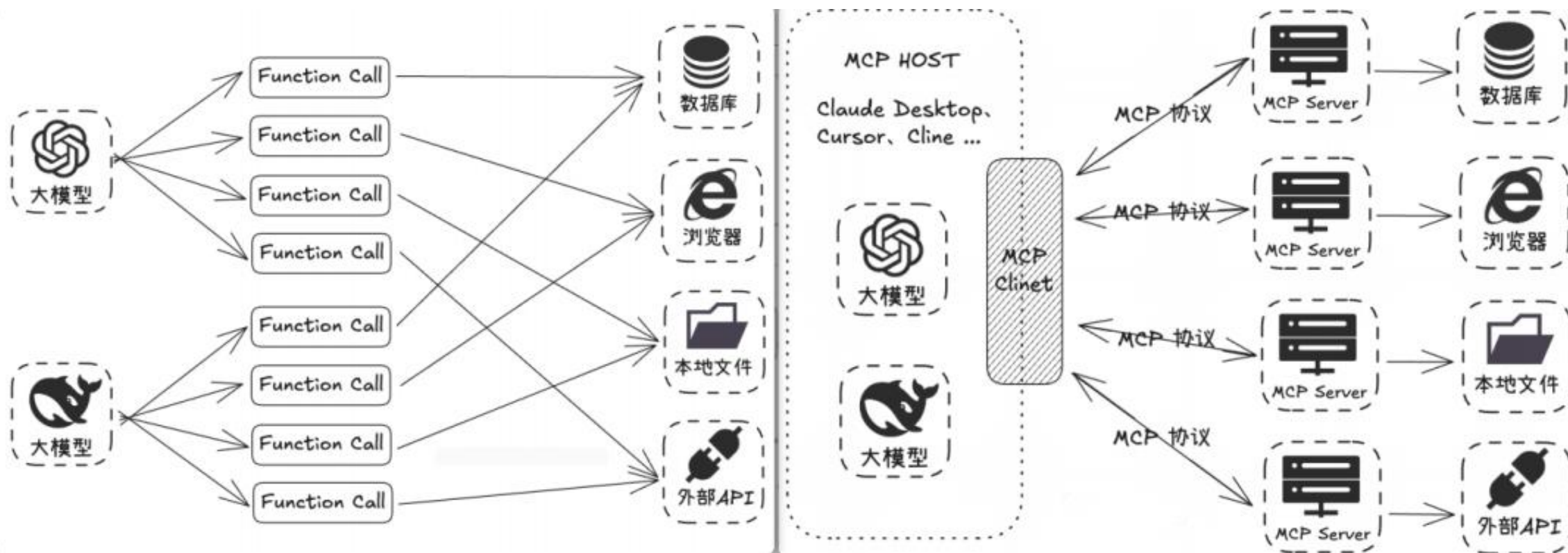
Model Context Protocol



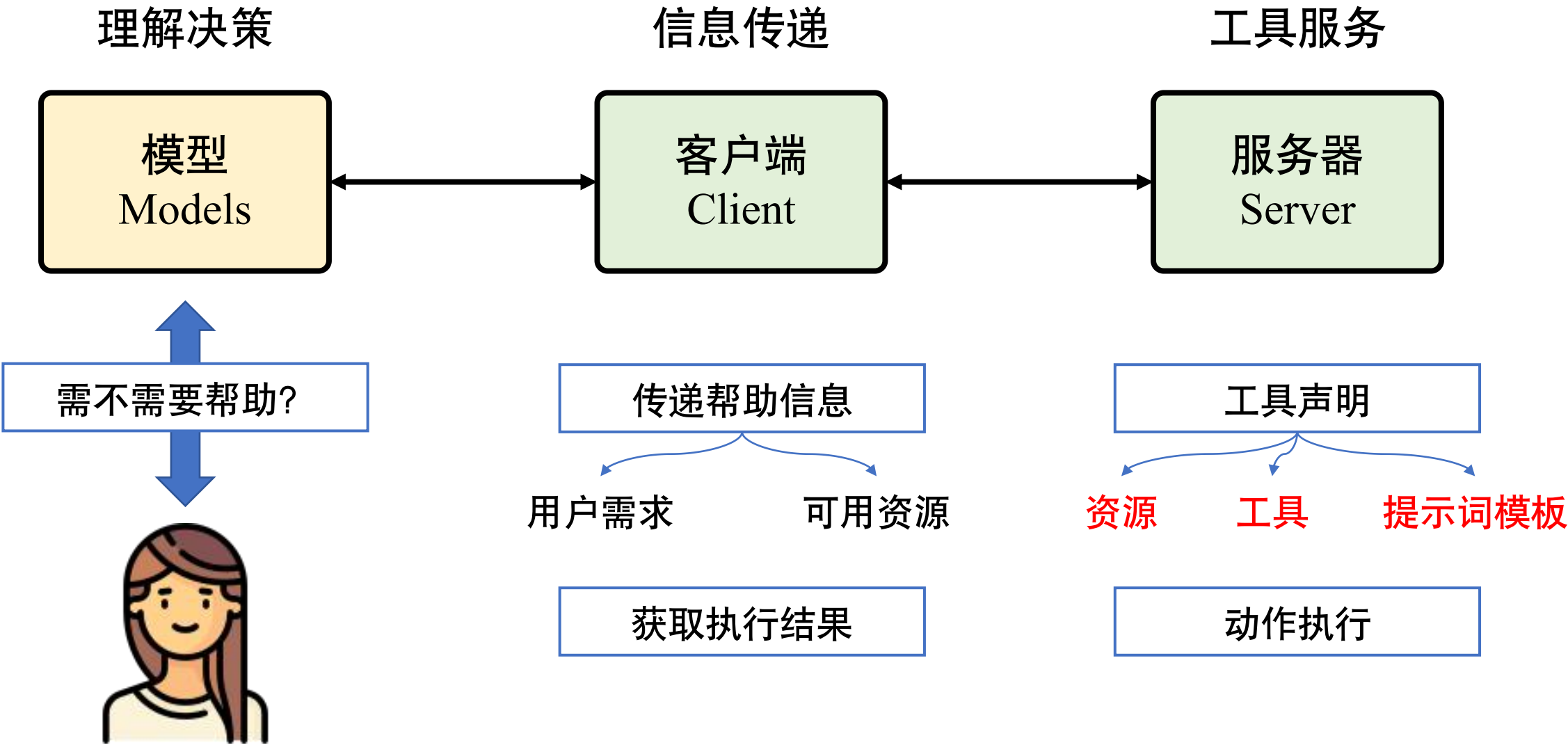
Model Context Protocol (MCP), clearly explained



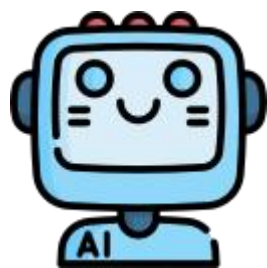
MCP ≠ Function Call Pro



2.1 MCP规定了什么角色



2.2 角色如何工作?



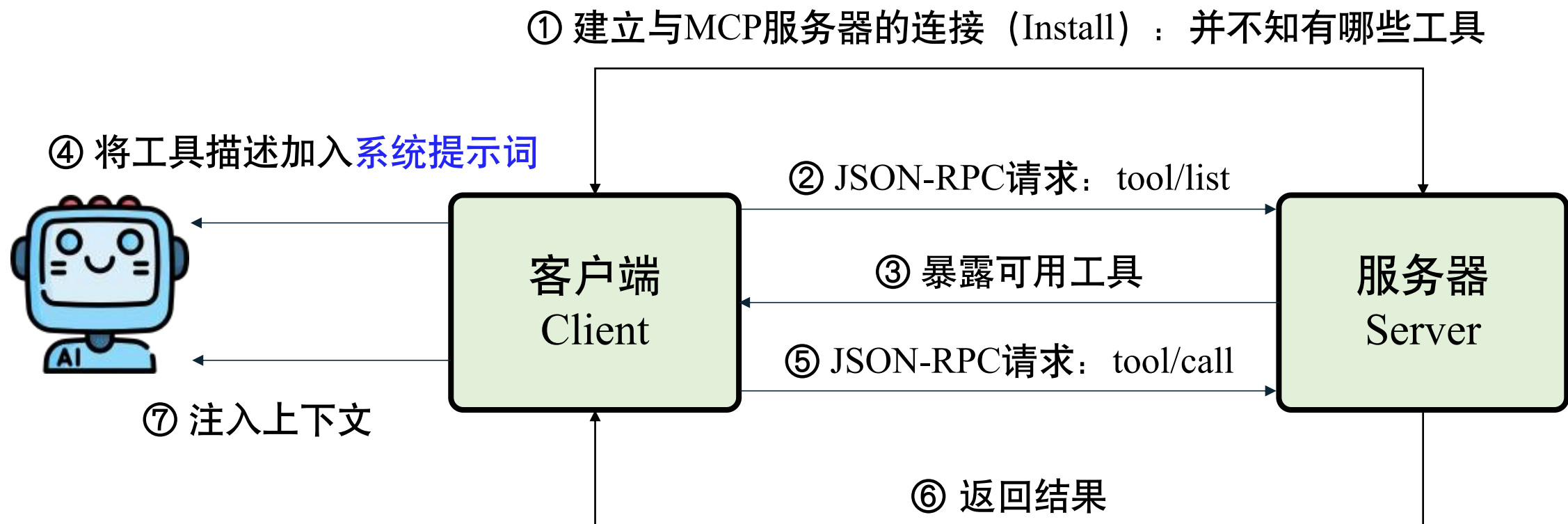

```
// MCP Client 发出: tools/list
// MCP Server 返回:
{
  "tools": [
    {
      "name": "submit_leave",
      "description": "向教务系统提交请假申请。当用户说'请假' '请个假'时调用。",
      "inputSchema": {
        "type": "object",
        "properties": {
          "reason": {"type": "string", "description": "请假原因"},
          "date": {"type": "string", "description": "请假日期, 格式YYYY-MM-DD"}
        },
        "required": ["reason"]
      }
    },
    {
      "name": "query_schedule",
      "description": "查询课表...",
      "inputSchema": { ... }
    },
    {
      "name": "query_grades",
      "description": "查询成绩...",
      "inputSchema": { ... }
    }
  ]
}
```

```
{
  "name": "get_weather",
  "description": "获取指定城市当前天气。当用户询问某地天气时调用。",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {"type": "string", "description": "城市名称, 如'北京'、'黄石'"}
    },
    "required": ["city"]
  }
}
```

两者几乎没有区别

所以, MCP并未修改工具调用的核心

2.2 角色如何工作?



好像有点问题?

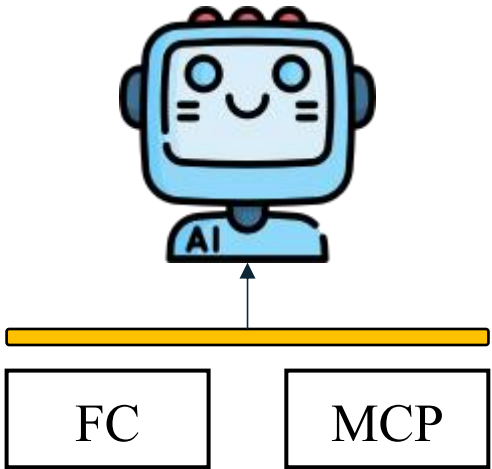
工具调用规则不统一

空间不足

记忆混乱

2.3 MCP对FC做了什么？

工具调用步骤	纯FC方法	MCP	做了什么？
① 注册工具	手动编写提示词	tools/list 自动发现	MCP 接管
② 用户提问	用户提示词	用户提示词	完全一样
③ 打包说明	系统提示词+工具描述	系统提示词+工具描述	完全一样
④ LLM 判据	工具描述语义匹配	工具描述语义匹配	完全一样
⑤ 执行	自定义函数调用	tools/call 服务器执行	MCP 接管



MCP对LLM透明

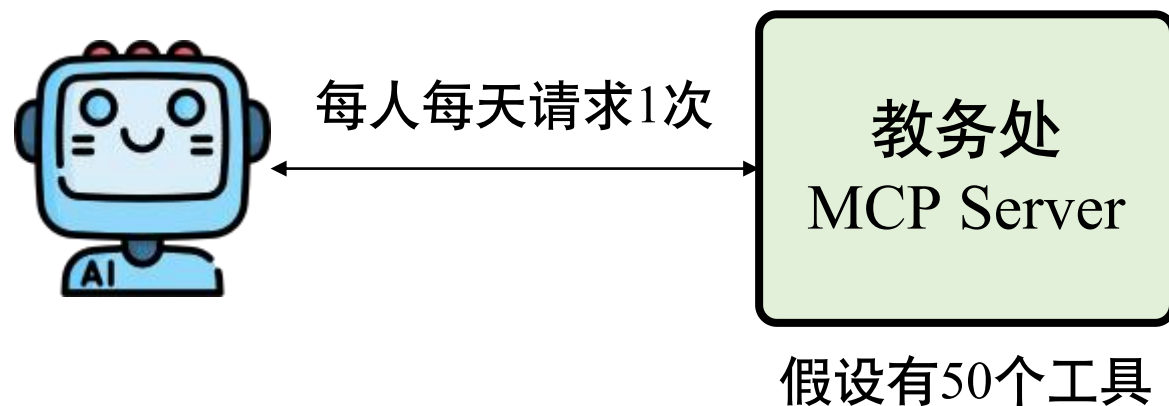


MCP内容改变不影响LLM



LLM对臃肿的MCP无能为力

2.4 算一笔经济账



湖北理工每年用于Token输入的
经费是多少？

submit_leave_request: 请假 (500 Tokens)

query_schedule: 查课表 (500 Tokens)

query_grades: 查成绩 (500 Tokens)

不相关的功能 1 (500 Tokens)

不相关的功能 2 (500 Tokens)

.....

不相关的功能 N (500 Tokens)

Chapter 4

Function Calling

让LLM长出手脚

模型上下文协议

命令行界面工具

3.1 Hermes中的CLI Tool定义

```
TERMINAL_TOOL_DESCRIPTION = """Execute shell commands on a Linux environment.
Filesystem usually persists between calls.
```

```
Do NOT use cat/head/tail to read files - use read_file instead.
Do NOT use grep/rg/find to search - use search_files instead.
Do NOT use sed/awk to edit files - use patch instead.
```

```
Reserve terminal for: builds, installs, git, processes, scripts, network,
package managers, and anything that needs a shell.
```

```
Foreground (default): Commands return INSTANTLY when done.
```

```
Background: Set background=true to get a session_id... ""
```

描述简洁

```
TERMINAL_SCHEMA = {
  "name": "terminal",
  "description": TERMINAL_TOOL_DESCRIPTION,
  "parameters": {
    "type": "object",
    "properties": {
      "command": {"type": "string",
                  "description": "The command to execute on the VM"},
      "background": {"type": "boolean", ...},
      "timeout": {"type": "integer", ...},
      "workdir": {"type": "string", ...},
    },
    "required": ["command"]
  }
}
```

参数简洁



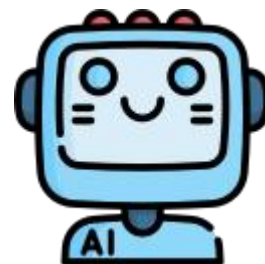
定义方式与FC没有区别，
本质也是一种工具形态。



比MCP更高效

再算一下“教务处”的账.....

账面会很好看



CLI: description (200 Tokens)

CLI: command (50 Tokens)

每人每天请求1次

每次平均调用5条命令



帖子



Peter Steinberger



@steipete



显示翻译

mcp were a mistake. bash is better.

上午2:06 · 2026年1月5日 · **506** 查看

3.2 CLI的高效并非一家之言

CLI Tools vs MCP: Better AI Agents With Less Context

22. FEBRUARY 2026 JANNIKREINHARD 1 COMMENT

任务：“列出所有不合规的 Intune 设备，并将其详细信息导出为 CSV 文件。”

CLI 效率提升近 35 倍

MCP approach (Microsoft Graph MCP Server):

Phase	Token Cost
Tool schema injection	~28,000 tokens
Agent reasoning + tool selection	~3,200 tokens
MCP call: list devices with filter	~1,800 tokens
MCP response parsing	~4,500 tokens
MCP call: get device details (per device)	~2,100 tokens × N
Total for 50 devices	~145,000 tokens

CLI approach:

Phase	Token Cost
Tool schema injection	0 tokens
Agent reasoning + command composition	~800 tokens
Shell command execution	~150 tokens
Output parsing	~3,200 tokens
Total for 50 devices	~4,150 tokens

MCP vs CLI for AI Agents: Which One Should You Use in 2026?

 Hiba Fathima · Apr 10, 2026

为什么 CLI 节省 Tokens?

- LLM在预训练时已见过海量的描述
- LLM比任何人都精通语法
- 定义一个Terminal调用所有Commands

Task Github 主要任务	CLI Tokens	MCP Tokens	Difference
Repo language and license	1,365	44,026	32x
PR details and review status	1,648	32,279	20x
Repo metadata and install	9,386	82,835	9x
Merged PRs by contributor	5,010	33,712	7x
Latest release and dependencies	8,750	37,402	4x

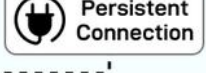
3.3 CLI有其劣势，而MCP有其优势

CLI vs. MCP: When to Use Each

Dimension	CLI	MCP	Winner
 Token Cost	command ~ 200 tokens	Schema Tools Fields Desc Auth ~ 44K tokens	✓ CLI No schema overhead to load
 Native Knowledge	gh aws curl git already in training data	{"tools": [...]} -----> [GPT icon] New schema at runtime	✓ CLI LLMs pretrained on CLI syntax
 Composability	gh jq grep One LLM call thanks to Unix pipes	[GPT icon] --> [cmd icon] --> [GPT icon] ... Multiple LLM calls needed	✓ CLI Unix pipes chain natively



复杂指令集
Complex Instruction Set Computing, CISC

 Multi-user Auth	 Shared Token can't revoke per user	 Per-user OAuth Revoke any one anytime	✓ MCP Each user gets own credentials
 Stateful Sessions	 cmd 1 -- New conn --> cmd 2 New TCP connection per command (~200ms overhead)	 Persistent Connection LLM call 1 --> LLM call 2 --> LLM call 3 ... Persistent server connection (~5ms overhead)	✓ MCP Persistent conn, reuses state
 Enterprise Governance	 ~/.bash_history Only record. No control	 audit  Revoke  Monitor	✓ MCP Audit trails + access control



精简指令集
Reduced Instruction Set Computing, RISC